Seck Mahmoune
Tindom Arol
Groupe 2

IUT Villetaneuse
Université Sorbonne Paris Nord

TP 1 : TUNNELS VPN
R401:INFRASTRUCTURES DE SÉCURITÉ

BUT 2 R&T
(GROUPE 2)

UNIVERSITÉ SORBONNE PARIS NORD

Réalisé par :
-Tindom Sila Arol
- Seck Mahmoune

Encadré par:
-Mr Ebongue Njoh

2025 - 2026

Seck Mahmoune
Tindom Arol
Groupe 2

IUT Villetaneuse
Université Sorbonne Paris Nord

TABLES DES MATIÈRES

Introduction 4
Exercice 1 : Travail préliminaire 4
Exercice 1 — Travail préliminaire 5
Question 1.1 à 1.3 5
Question 1.4 6
Exercice 2 — Configuration des interfaces 6
Question 2.1 6
Question 2.2 7
Question 2.3 8
Question 2.4 9
Exercice 3 : Mise en place de la translation d'adresse 11
Question 3.1 & Question 2 : 11
Question 3.3 : 11
Question 3.4 12
Questions 3.6 & 3.7 13
Question 3.8 14
Question 3.9 15
Question 3.10 16
Exercice 4 — Création d'un tunnel VPN de niveau 2 17
Questions 4.1 & 4.2 17
Question 4.3 18
Question 4.4 19
Question 4.5 20
Question 4.6 21
Question 4.7 21
Question 4.8 22
Question 4.1 23
Question 4.9 23
Question 4.10 24
Exercice 5 — Création d'un tunnel VPN de niveau 3 26
Questions 5.1 & 5.2 26
Question 5.3 27
Question 5.4 27
Question 5.5 28
Question 5.6 29
Question 5.7 30
Question 5.8 31
Question 5.1 31
Question 5.9 32

Question 5.2 33
Question 5.3 : 33
Question 5.10 34
Exercice 6 : Mise en place de la sécurité (Chiffrement) 34
Question 6.1 35
Question 6.2 35
Question 6.3 36
Question 6.4 37
Question 6.5 38
Question 6.6 (Conclusion de l'Analyse) 38
Conclusion Générale du TP 39
1. Du Pontage au Routage (Niveau 2 vs Niveau 3) 39
2. L'Importance du Routage IP 39
3. La Sécurisation des Données 40

Introduction

L'objectif de ce TP est de mettre en œuvre et d'analyser des techniques d'interconnexion de réseaux privés distants via un réseau public non sécurisé. À travers l'outil *Marionnet*, nous étudions la mise en place de tunnels VPN de niveau 2 (pontage Ethernet) et de niveau 3 (routage IP), ainsi que les mécanismes de translation d'adresses (NAT) et la sécurisation des échanges par chiffrement symétrique.

Exercice 1 : Travail préliminaire

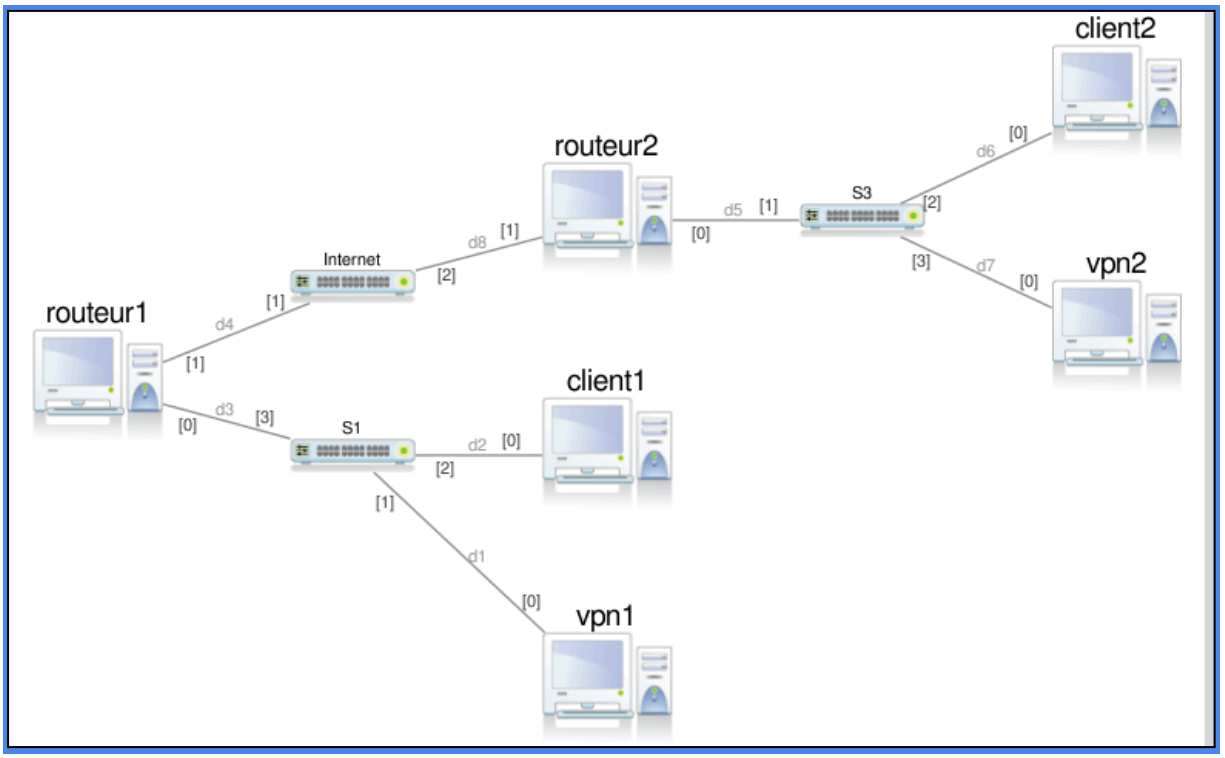
Objectif : Préparer la plateforme de simulation et construire la topologie réseau.

Question 1.1 à 1.3

Démarche : Nous avons téléchargé le fichier de projet `vpn.mar`, l'avons chargé dans l'environnement *Marionnet* et avons ajouté manuellement les composants manquants (Routeurs, Switchs et Machines) pour reproduire le schéma fourni dans l'énoncé.

Résultat :

Seck Mahmoune
Tindom Arol
Groupe 2



Explication des résultats : La maquette est prête. On distingue deux domaines privés (LAN1 et LAN2) séparés par un domaine public simulant Internet (réseau 1.2.3.0/24).

Question 1.4

Démarche : Nous avons démarré l'ensemble des machines virtuelles de la session.
Explication des résultats : Toutes les fenêtres de terminal sont apparues, nous permettant d'interagir avec chaque équipement en tant qu'administrateur (root).

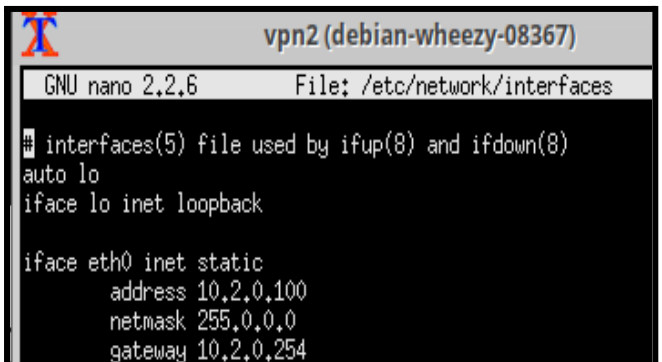
Exercice 2 — Configuration des interfaces

Objectif : Définir l'adressage IP et activer le transit des paquets sur les routeurs.

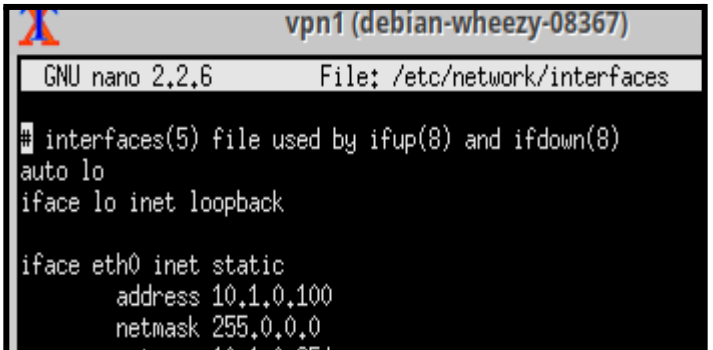
Question 2.1

Démarche : Sur chaque machine, nous avons édité le fichier `/etc/network/interfaces`.Pour `client1`, nous avons configuré l'adresse `10.1.0.1` avec la passerelle `10.1.0.254`.

vnp 2 :



vnp 1 :



Seck Mahmoune
Tindom Arol
Groupe 2

routeur 1 :

```
GNU nano 2.2.6        File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.254
        netmask 255.0.0.0

iface eth1 inet static
        address 1.2.3.2
        netmask 255.255.255.0
```

routeur 2 :

```
GNU nano 2.2.6        File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.254
        netmask 255.0.0.0

iface eth1 inet static
        address 1.2.3.1
        netmask 255.255.255.0
```

client 1 :

```
GNU nano 2.2.6        File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.1
        netmask 255.0.0.0
```

client 2 :

```
GNU nano 2.2.6        File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.2.0.1
        netmask 255.0.0.0
```

Explication des résultats : L'adressage est désormais persistant. Chaque machine possède une identité IP conforme au plan d'adressage du TP.

Question 2.2

Démarche : Nous avons utilisé la commande `ifup` sur les interfaces concernées (ex: `ifup eth0`).

vpn 1 :

```
[0 root@vpn1 ~]$ ifup eth0
[0 root@vpn1 ~]$
```

Seck Mahmoune
Tindom Arol
Groupe 2

vpn 2 :

```
[0 root@vpn2 ~]$ ifup eth0
[0 root@vpn2 ~]$
```

routeur 1 :

```
[0 root@routeur1 ~]$ ifup eth0
[0 root@routeur1 ~]$ ifup eth1
[0 root@routeur1 ~]$
```

routeur 2 :

```
[0 root@routeur2 ~]$ ifup eth0
[0 root@routeur2 ~]$ ifup eth1
[0 root@routeur2 ~]$
```

client1 :

```
[0 root@client1 ~]$ ifup eth0
[0 root@client1 ~]$
```

client 2 :

```
[0 root@client2 ~]$ ifup eth0
[0 root@client2 ~]$
```

Explication des résultats : Les interfaces sont "Up", permettant la communication physique et logique sur les liens connectés.

Question 2.3

Démarche : Sur les deux routeurs, nous avons exécuté la commande `sysctl net.ipv4.ip_forward=1`.

routeur 1 :

```
[0 root@routeur1 ~]$ sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[0 root@routeur1 ~]$
```

routeur 2 :

```
[0 root@routeur2 ~]$ sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[0 root@routeur2 ~]$
```

Seck Mahmoune
Tindom Arol
Groupe 2

Explication des résultats : Cette action autorise le noyau Linux à transférer des paquets entre ses interfaces **eth0** et **eth1**, transformant la machine simple en routeur actif.

Question 2.4

Démarche : Nous avons effectué des tests de connectivité avec la commande `ping` entre les clients et leurs passerelles respectives.

Ping dans le lan 1

client1 :

```
[0 root@client1 ~]$ ping -c3 10.1.0.254
PING 10.1.0.254 (10.1.0.254) 56(84) bytes of data.
64 bytes from 10.1.0.254: icmp_req=1 ttl=64 time=1.68 ms
64 bytes from 10.1.0.254: icmp_req=2 ttl=64 time=3.02 ms
64 bytes from 10.1.0.254: icmp_req=3 ttl=64 time=1.79 ms

--- 10.1.0.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 1.686/2.167/3.023/0.606 ms
[0 root@client1 ~]$ ping -c3 10.1.0.100
PING 10.1.0.100 (10.1.0.100) 56(84) bytes of data.
64 bytes from 10.1.0.100: icmp_req=1 ttl=64 time=1.45 ms
64 bytes from 10.1.0.100: icmp_req=2 ttl=64 time=2.39 ms
64 bytes from 10.1.0.100: icmp_req=3 ttl=64 time=1.64 ms

--- 10.1.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 1.452/1.829/2.395/0.410 ms
[0 root@client1 ~]$
```

vpn1 :

```
[0 root@vpn1 ~]$ ping -c3 10.1.0.254
PING 10.1.0.254 (10.1.0.254) 56(84) bytes of data.
64 bytes from 10.1.0.254: icmp_req=1 ttl=64 time=1.57 ms
64 bytes from 10.1.0.254: icmp_req=2 ttl=64 time=2.99 ms
64 bytes from 10.1.0.254: icmp_req=3 ttl=64 time=2.40 ms

--- 10.1.0.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 1.570/2.322/2.996/0.587 ms
[0 root@vpn1 ~]$ ping -c3 10.1.0.1
PING 10.1.0.1 (10.1.0.1) 56(84) bytes of data.
64 bytes from 10.1.0.1: icmp_req=1 ttl=64 time=1.42 ms
64 bytes from 10.1.0.1: icmp_req=2 ttl=64 time=2.65 ms
64 bytes from 10.1.0.1: icmp_req=3 ttl=64 time=1.55 ms

--- 10.1.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 1.426/1.880/2.659/0.553 ms
[0 root@vpn1 ~]$
```

routeur 1 :

```
[0 root@routeur1 ~]$ ping -c3 10.1.0.1
PING 10.1.0.1 (10.1.0.1) 56(84) bytes of data.
64 bytes from 10.1.0.1: icmp_req=1 ttl=64 time=0.881 ms
64 bytes from 10.1.0.1: icmp_req=2 ttl=64 time=2.71 ms
64 bytes from 10.1.0.1: icmp_req=3 ttl=64 time=1.89 ms

--- 10.1.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 0.881/1.829/2.712/0.750 ms
[0 root@routeur1 ~]$ ping -c3 10.1.0.100
PING 10.1.0.100 (10.1.0.100) 56(84) bytes of data.
64 bytes from 10.1.0.100: icmp_req=1 ttl=64 time=1.43 ms
64 bytes from 10.1.0.100: icmp_req=2 ttl=64 time=1.11 ms
64 bytes from 10.1.0.100: icmp_req=3 ttl=64 time=0.305 ms
```

Seck Mahmoune
Tindom Arol
Groupe 2

IUT Villetaneuse
Université Sorbonne Paris Nord

PING DANS LE LAN 2

vpn2 :

```
[0 root@vpn2 ~]$ ping -c3 10.2.0.254
PING 10.2.0.254 (10.2.0.254) 56(84) bytes of data.
64 bytes from 10.2.0.254: icmp_req=1 ttl=64 time=1.76 ms
64 bytes from 10.2.0.254: icmp_req=2 ttl=64 time=2.76 ms
64 bytes from 10.2.0.254: icmp_req=3 ttl=64 time=2.29 ms

--- 10.2.0.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2016ms
rtt min/avg/max/mdev = 1.761/2.273/2.764/0.409 ms
[0 root@vpn2 ~]$ ping -c3 10.2.0.1
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=64 time=1.39 ms
64 bytes from 10.2.0.1: icmp_req=2 ttl=64 time=2.60 ms
64 bytes from 10.2.0.1: icmp_req=3 ttl=64 time=1.41 ms

--- 10.2.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 1.398/1.804/2.601/0.563 ms
[0 root@vpn2 ~]$
```

routeur 2 :

```
[0 root@routeur2 ~]$ ping -c3 10.2.0.100
PING 10.2.0.100 (10.2.0.100) 56(84) bytes of data.
64 bytes from 10.2.0.100: icmp_req=1 ttl=64 time=1.08 ms
64 bytes from 10.2.0.100: icmp_req=2 ttl=64 time=1.87 ms
64 bytes from 10.2.0.100: icmp_req=3 ttl=64 time=1.59 ms

--- 10.2.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2012ms
rtt min/avg/max/mdev = 1.087/1.517/1.872/0.326 ms
[0 root@routeur2 ~]$ ping -c3 10.2.0.1
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=64 time=1.72 ms
64 bytes from 10.2.0.1: icmp_req=2 ttl=64 time=1.03 ms
64 bytes from 10.2.0.1: icmp_req=3 ttl=64 time=2.17 ms

--- 10.2.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.031/1.642/2.172/0.470 ms
[0 root@routeur2 ~]$
```

client 2 :

```
[0 root@client2 ~]$ ping -c3 10.2.0.254
PING 10.2.0.254 (10.2.0.254) 56(84) bytes of data.
64 bytes from 10.2.0.254: icmp_req=1 ttl=64 time=1.33 ms
64 bytes from 10.2.0.254: icmp_req=2 ttl=64 time=2.15 ms
64 bytes from 10.2.0.254: icmp_req=3 ttl=64 time=2.29 ms

--- 10.2.0.254 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2013ms
rtt min/avg/max/mdev = 1.334/1.924/2.290/0.424 ms
[0 root@client2 ~]$ ping -c3 10.2.0.100
PING 10.2.0.100 (10.2.0.100) 56(84) bytes of data.
64 bytes from 10.2.0.100: icmp_req=1 ttl=64 time=1.25 ms
64 bytes from 10.2.0.100: icmp_req=2 ttl=64 time=0.711 ms
64 bytes from 10.2.0.100: icmp_req=3 ttl=64 time=1.42 ms

--- 10.2.0.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2009ms
rtt min/avg/max/mdev = 0.711/1.129/1.421/0.303 ms
```

Seck Mahmoune
Tindom Arol
Groupe 2

Explication des résultats : Le pings entre les machines du LAN1 et les pings entre les machines du LAN2 réussissent avec un TTL de 64. Cela prouve que le réseau local fonctionne, mais la communication inter-LAN reste impossible à ce stade.

Exercice 3 : Mise en place de la translation d'adresse

Question 3.1 & Question 2 :

Démarche : Nous avons configuré `iptables` sur les routeurs pour effectuer du Source NAT (ex: `iptables -t nat -A POSTROUTING -o eth1 -j SNAT --to-source 1.2.3.1`).

Résultat :

```
[0 root@routeur1 ~]$ iptables -t nat -A POSTROUTING -o eth1 -j SNAT --to-source
1.2.3.1
[0 root@routeur1 ~]$
```

```
[0 root@routeur2 ~]$ iptables -t nat -A POSTROUTING -o eth1 -j SNAT --to-source
1.2.3.2
[0 root@routeur2 ~]$
```

Explication des résultats : Le routeur remplace désormais l'adresse IP source privée des paquets sortants par sa propre adresse publique.

Question 3.3 :

Démarche : Test de ping depuis `vpn1` vers l'interface publique de `routeur2` (`1.2.3.2`). et de `vpn2` vers `routeur1(1.2.3.1)`.

Résultat :

Vpn 1 -> routeur2

```
[0 root@vpn1 ~]$ ping -c2 1.2.3.2
PING 1.2.3.2 (1.2.3.2) 56(84) bytes of data.
64 bytes from 1.2.3.2: icmp_req=1 ttl=64 time=1.93 ms
64 bytes from 1.2.3.2: icmp_req=2 ttl=64 time=2.36 ms

--- 1.2.3.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1005ms
rtt min/avg/max/mdev = 1.935/2.150/2.365/0.215 ms
[0 root@vpn1 ~]$
```

Vpn 2 -> routeur1

```
[1 root@vpn2 ~]$ ping -c2 1.2.3.1
PING 1.2.3.1 (1.2.3.1) 56(84) bytes of data.
64 bytes from 1.2.3.1: icmp_req=1 ttl=63 time=2.50 ms
64 bytes from 1.2.3.1: icmp_req=2 ttl=63 time=3.79 ms

--- 1.2.3.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 2.500/3.146/3.793/0.648 ms
[0 root@vpn2 ~]$
```

Explication des résultats : Le ping aboutit. Grâce au SNAT, une machine du réseau privé peut désormais envoyer des paquets sur le réseau public.

Question 3.4

Démarche : Afin de vérifier que la translation d'adresse source (SNAT) est effective, nous avons utilisé l'outil de capture de paquets `tcpdump` sur le **routeur2** (le point de réception du trafic provenant du réseau public). Nous avons exécuté la commande suivante sur le terminal du routeur2 : `tcpdump -i eth1 icmp` Pendant que cette capture était active, nous avons lancé une requête `ping` depuis la machine **vpn1** vers l'adresse IP publique du routeur2 (`1.2.3.2`).

Résultats :

```
[0 root@vpn1 ~]$ ping -c2 1.2.3.2
PING 1.2.3.2 (1.2.3.2) 56(84) bytes of data.
64 bytes from 1.2.3.2: icmp_req=1 ttl=63 time=2.76 ms
64 bytes from 1.2.3.2: icmp_req=2 ttl=63 time=3.06 ms

--- 1.2.3.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 2.767/2.915/3.064/0.158 ms
[0 root@vpn1 ~]$
```

```
[0 root@routeur2 ~]$ tcpdump -i eth1 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 65535 bytes
16:26:31.548052 IP 1.2.3.1 > 1.2.3.2: ICMP echo request, id 1723, seq 1, length
64
16:26:31.548099 IP 1.2.3.2 > 1.2.3.1: ICMP echo reply, id 1723, seq 1, length 64
16:26:32.555106 IP 1.2.3.1 > 1.2.3.2: ICMP echo request, id 1723, seq 2, length
64
16:26:32.555137 IP 1.2.3.2 > 1.2.3.1: ICMP echo reply, id 1723, seq 2, length 64
^C
4 packets captured
4 packets received by filter
0 packets dropped by kernel
[0 root@routeur2 ~]$
```

Explication des résultats : L'analyse des paquets capturés montre que les requêtes ICMP arrivant sur l'interface `eth1` du routeur2 ont pour adresse IP source **1.2.3.1** (l'adresse publique du routeur1) et non **10.1.0.100** (l'adresse privée réelle du vpn1).

Ce résultat confirme le succès de la configuration du SNAT sur le routeur1 : le paquet a été intercepté lors de sa sortie, son adresse source privée a été masquée et remplacée par l'adresse publique du routeur, permettant ainsi au trafic d'être routable sur le réseau extérieur.

Question 3.5 :

Démarche : Après avoir lancé la capture avec `tcpdump` sur le routeur2 lors de la question précédente, nous avons analysé précisément les en-têtes des paquets ICMP (Echo Request) reçus sur l'interface publique `eth1`. L'objectif était de comparer l'adresse IP source réelle de l'émetteur (vpn1) avec l'adresse IP source vue par le destinataire après le passage par le routeur1.

Résultats :

```
[0 root@vpn2 ~]$ ping -c2 1.2.3.1
PING 1.2.3.1 (1.2.3.1) 56(84) bytes of data.
64 bytes from 1.2.3.1: icmp_req=1 ttl=63 time=1.59 ms
64 bytes from 1.2.3.1: icmp_req=2 ttl=63 time=3.57 ms

--- 1.2.3.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 1.593/2.584/3.576/0.992 ms
[0 root@vpn2 ~]$
```

```
[0 root@routeur1 ~]$ tcpdump -i eth1 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 65535 bytes
16:27:33.181533 IP 1.2.3.2 > 1.2.3.1: ICMP echo request, id 1728, seq 1, length
64
16:27:33.181582 IP 1.2.3.1 > 1.2.3.2: ICMP echo reply, id 1728, seq 1, length 64
16:27:34.190400 IP 1.2.3.2 > 1.2.3.1: ICMP echo request, id 1728, seq 2, length
64
16:27:34.190425 IP 1.2.3.1 > 1.2.3.2: ICMP echo reply, id 1728, seq 2, length 64
```

Explication des résultats : L'analyse de la capture montre que le champ "Source IP" du paquet contient l'adresse **1.2.3.1**. Or, nous savons que la machine qui a généré le ping est le `vpn1` avec l'adresse **10.1.0.100**.

Ce résultat prouve que le mécanisme de **SNAT** (Source NAT) configuré sur le routeur1 fonctionne parfaitement : le routeur a intercepté le paquet sortant du LAN1 et a substitué l'adresse privée (non routable sur Internet) par son adresse publique. Cela permet au `vpn1` de communiquer avec l'extérieur tout en restant masqué derrière la passerelle, ce qui est une étape indispensable avant d'établir le tunnel VPN à travers le réseau public.

Questions 3.6 & 3.7

Seck Mahmoune
Tindom Arol
Groupe 2

Démarche : Afin de permettre l'établissement du tunnel VPN à travers les routeurs, nous avons mis en place une règle de **DNAT** (Destination NAT), aussi appelée redirection de port (Port Forwarding). Sur le **routeur1**, nous avons utilisé la commande `iptables` pour rediriger tout le trafic UDP arrivant sur son interface publique (`eth1`) sur le port **1194** vers l'adresse IP privée du serveur **vpn1** (`10.1.0.100`). Nous avons appliqué une configuration similaire sur le **routeur2** pour rediriger le trafic vers le serveur **vpn2** (`10.2.0.100`).

Résultats :

```
[0 root@routeur1 ~]$ iptables -t nat -A PREROUTING -i eth1 -p udp --dport 1194 -
j DNAT --to 10.1.0.100
[0 root@routeur1 ~]$
```

```
[0 root@routeur2 ~]$ iptables -t nat -A PREROUTING -i eth1 -p udp --dport 1194 -
j DNAT --to 10.2.0.100
[0 root@routeur2 ~]$
```

Explication des résultats : L'application de ces règles dans la chaîne `PREROUTING` de la table `nat` permet de rendre les serveurs VPN, pourtant situés dans des réseaux privés, "visibles" depuis l'extérieur.

Concrètement, lorsqu'un serveur VPN tente d'initier une connexion vers l'adresse publique du routeur distant, ce dernier ne rejette plus le paquet : il le transmet automatiquement à la machine interne concernée. Sans cette redirection, le trafic VPN serait bloqué par les routeurs dès son entrée, rendant impossible la création du tunnel sécurisé entre le LAN1 et le LAN2.

Question 3.8

Démarche : Après avoir configuré la redirection de port (DNAT) sur le routeur, nous sommes passés à la mise en service du serveur VPN. Sur la machine **vpn2**, nous avons exécuté la commande suivante pour lancer OpenVPN en mode "écoute" : `openvpn --remote 1.2.3.1 --dev tap1` Cette commande indique à **vpn2** de se préparer à recevoir une connexion provenant de l'adresse publique du routeur distant, en utilisant une interface réseau virtuelle de type `tap`.

Résultat :

```
[0 root@vpn2 ~]$ openvpn --dev tap1
Wed Feb 11 16:33:43 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [EPOLL] [PKCS
11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec
1 2014
Wed Feb 11 16:33:43 2026 IMPORTANT: OpenVPN's default port number is now 1194, b
ased on an official port number assignment by IANA.  OpenVPN 2.0-beta16 and earl
ier used 5000 as the default port.
Wed Feb 11 16:33:43 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or hig
her to call user-defined scripts or executables
Wed Feb 11 16:33:43 2026 ******* WARNING *******: all encryption and authenticat
ion features disabled -- all data will be tunnelled as cleartext
Wed Feb 11 16:33:43 2026 TUN/TAP device tap1 opened
Wed Feb 11 16:33:43 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 16:33:43 2026 UDPv4 link remote: [undef]
```

Explication des résultats : Le lancement du processus OpenVPN permet d'ouvrir le port UDP 1194 sur la machine **vpn2**. Désormais, grâce à la règle de DNAT précédemment établie sur le **routeur2**, tout paquet arrivant d'Internet sur le port 1194 sera directement transmis à ce processus.

L'utilisation de l'option `--dev tap1` est ici cruciale : elle crée une interface de niveau 2 (liaison de données) qui permettra, dans les étapes suivantes, de ponter le trafic Ethernet du LAN2 avec celui du tunnel. À ce stade, le serveur est en attente de la "poignée de main" (handshake) provenant du client **vpn1**.

Question 3.9

Démarche : Après avoir lancé le serveur sur `vpn2` à la question précédente, nous avons effectué un nouveau test de balayage de port (scan) depuis le **routeur1** à l'aide de l'utilitaire **nmap**. La commande utilisée était : `nmap -sU -p 1194 1.2.3.2` L'objectif était de confirmer que le service OpenVPN était désormais actif derrière le routeur et prêt à accepter des paquets via la redirection de port (DNAT).

Résultat :

```
[0 root@routeur1 ~]$ nmap -sU -p 1194 1.2.3.2

Starting Nmap 6.00 ( http://nmap.org ) at 2026-02-11 16:34 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
 Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 1.2.3.2
Host is up (0.0013s latency).
PORT     STATE         SERVICE
1194/udp open|filtered openvpn
MAC Address: 02:04:06:0F:8F:66 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds
[0 root@routeur1 ~]$
```

Explication des résultats : Le résultat du scan affiche l'état `open|filtered` pour le port 1194/udp.

Cet état confirme deux choses essentielles :

1. La règle de **DNAT** sur le routeur2 est correctement configurée, car elle laisse passer les paquets UDP vers l'hôte interne.
2. Le processus **OpenVPN** est bien en cours d'exécution sur le serveur `vpn2` et "écoute" sur ce port.

Contrairement à un port TCP qui répondrait par un "SYN-ACK", le protocole UDP est sans connexion ; l'absence de message d'erreur ICMP (port inaccessible) indique au scanner que le paquet a bien atteint sa destination. La chaîne de liaison entre le réseau public et le service privé est donc totalement opérationnelle.

Seck Mahmoune
Tindom Arol
Groupe 2

Question 3.10

Démarche : Après avoir testé le fonctionnement du tunnel, nous avons procédé à l'arrêt du serveur OpenVPN sur la machine **vpn2** (en utilisant `Ctrl+C`). Pour vérifier l'impact de cet arrêt sur la visibilité du service depuis l'extérieur, nous avons relancé un scan **nmap** depuis le **routeur1** vers l'adresse publique du **routeur2** (`1.2.3.2`).

Résultats :

```
[0 root@vpn2 ~]$ openvpn --dev tap1
Wed Feb 11 16:33:43 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [EPOLL] [PKCS
11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec
1 2014
Wed Feb 11 16:33:43 2026 IMPORTANT: OpenVPN's default port number is now 1194, b
ased on an official port number assignment by IANA.  OpenVPN 2.0-beta16 and earl
ier used 5000 as the default port.
Wed Feb 11 16:33:43 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or hig
her to call user-defined scripts or executables
Wed Feb 11 16:33:43 2026 ******* WARNING *******: all encryption and authenticat
ion features disabled -- all data will be tunnelled as cleartext
Wed Feb 11 16:33:43 2026 TUN/TAP device tap1 opened
Wed Feb 11 16:33:43 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 16:33:43 2026 UDPv4 link remote: [undef]
^CWed Feb 11 16:35:57 2026 event_wait : Interrupted system call (code=4)
Wed Feb 11 16:35:57 2026 SIGINT[hard,] received, process exiting
```

```
[0 root@routeur1 ~]$ nmap -sU -p 1194 1.2.3.2

Starting Nmap 6.00 ( http://nmap.org ) at 2026-02-11 16:36 UTC
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
 Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 1.2.3.2
Host is up (0.00076s latency).
PORT     STATE  SERVICE
1194/udp closed openvpn
MAC Address: 02:04:06:0F:8F:66 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
[0 root@routeur1 ~]$
```

Explication des résultats : Contrairement au test précédent, le résultat du scan indique désormais que le port 1194/udp est **"closed"** (fermé).

Cela s'explique par le fait que, bien que la règle de **DNAT** sur le routeur2 soit toujours active et redirige bien les paquets vers le serveur `vpn2`, il n'y a plus aucun processus (serveur OpenVPN) à l'écoute sur ce port côté serveur. La machine `vpn2` renvoie donc un message d'erreur ICMP "Port Unreachable" au routeur1. Ce test confirme deux points :

1. Le service est bien arrêté.
2. Notre diagnostic précédent était correct : c'est bien la présence du serveur actif qui rendait le port "ouvert" à travers le NAT.

Seck Mahmoune
Tindom Arol
Groupe 2

Exercice 4 — Création d'un tunnel VPN de niveau 2

Objectif de l'exercice : Mettre en œuvre un tunnel de niveau liaison (TAP) et configurer un pontage (bridging) pour fusionner les deux LANs distants en un seul segment Ethernet logique, comme s'ils étaient sur le même switch.

Questions 4.1 & 4.2

Démarche : Pour interconnecter les deux LANs, nous avons configuré une interface de pontage nommée `br0` sur les serveurs **vpn1** et **vpn2**. Au lieu de créer le pont manuellement avec des commandes temporaires, nous avons vérifié ou édité la configuration réseau pour que l'interface `br0` regroupe l'interface physique `eth0` (côté LAN) et l'interface virtuelle `tap1` (côté tunnel). Nous avons ensuite activé cette interface pour qu'elle porte l'adresse IP du serveur VPN dans son LAN respectif.

Résultats :

```
  GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.100
        netmask 255.0.0.0
        gateway 10.1.0.254

iface br0 inet static
        bridge_ports eth0 tap1
        address 10.1.0.100
        netmask 255.0.0.0
        gateway 10.1.0.254
```

```
  GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.2.0.100
        netmask 255.0.0.0
        gateway 10.2.0.254

iface br0 inet static
        bridge_ports eth0 tap1
        address 10.2.0.100
        netmask 255.0.0.0
        gateway 10.2.0.254
```

Explication des résultats : L'activation de l'interface `br0` permet de faire fonctionner le serveur VPN comme un commutateur (switch) virtuel.

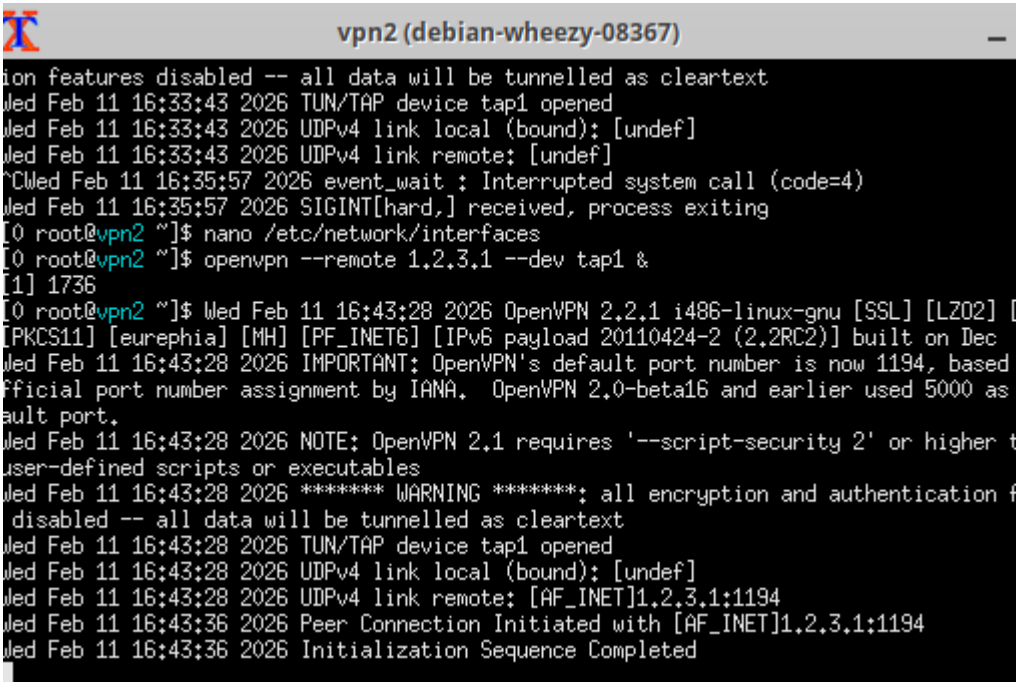
Le résultat de la commande (`ip link`) confirme que `eth0` et `tap1` sont liées au sein du même pont. Cela signifie que toute trame Ethernet reçue sur le réseau local est capable de basculer vers l'interface du tunnel. Cette configuration est la base du VPN de niveau 2 : elle permet d'abolir la frontière physique entre les deux sites distants pour n'en faire qu'un seul réseau logique.

Question 4.3

Démarche : Une fois le pontage (`br0`) prêt sur les deux passerelles, nous avons lancé le service OpenVPN pour monter le tunnel de niveau 2. Nous avons exécuté les commandes suivantes :

- Sur **vpn1** : `openvpn --remote 1.2.3.2 --dev tap1`
- Sur **vpn2** : `openvpn --remote 1.2.3.1 --dev tap1` L'utilisation de l'option `--dev tap1` est ici fondamentale car elle permet à OpenVPN de s'appuyer sur l'interface virtuelle déjà intégrée à notre pont réseau.

Resultat :



```
[0 root@vpn1 ~]$ openvpn --remote 1.2.3.2 --dev tap1 &
[1] 1729
[0 root@vpn1 ~]$ Wed Feb 11 16:42:15 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZ
O2] [EPOLL] [PKCS11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2
)] built on Dec  1 2014
Wed Feb 11 16:42:15 2026 IMPORTANT: OpenVPN's default port number is now 1194, b
ased on an official port number assignment by IANA.  OpenVPN 2.0-beta16 and earl
ier used 5000 as the default port.
Wed Feb 11 16:42:15 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or hig
her to call user-defined scripts or executables
Wed Feb 11 16:42:15 2026 ******* WARNING *******: all encryption and authenticat
ion features disabled -- all data will be tunnelled as cleartext
Wed Feb 11 16:42:15 2026 TUN/TAP device tap1 opened
Wed Feb 11 16:42:15 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 16:42:15 2026 UDPv4 link remote: [AF_INET]1.2.3.2:1194
Wed Feb 11 16:42:25 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:42:35 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:42:45 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:42:55 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:43:06 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:43:16 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:43:26 2026 read UDPv4 [ECONNREFUSED]: Connection refused (code=111
)
Wed Feb 11 16:43:36 2026 Peer Connection Initiated with [AF_INET]1.2.3.2:1194
Wed Feb 11 16:43:37 2026 Initialization Sequence Completed
[0 root@vpn1 ~]$
```

Explication des résultats : L'apparition du message « *Initialization Sequence Completed* » sur les deux terminaux confirme que la connexion entre les deux serveurs est établie à travers le réseau public.

Contrairement à un tunnel classique, ici OpenVPN ne crée pas seulement une interface de communication, il active le "câble virtuel" qui relie les deux commutateurs `br0`. Le tunnel est désormais capable de transporter des trames Ethernet brutes. Le fait que le processus reste actif sans erreur indique que les règles de NAT (DNAT/SNAT) configurées précédemment sur les routeurs laissent bien passer le flux VPN sur le port 1194.

Question 4.4

Démarche : Une fois le pont réseau configuré, nous avons procédé à l'activation des interfaces **br0** sur les serveurs **vpn1** et **vpn2**. Pour ce faire, nous avons utilisé la commande : `ifup br0` (ou `ip link set br0 up`) Cette étape permet de monter l'interface logique qui regroupe désormais `eth0` et `tap1`. Nous avons également vérifié que l'adresse IP du serveur était bien assignée à `br0` pour permettre la communication avec le reste du LAN.

Résultats :

```
Wed Feb 11 16:45:37 2026 initialization sequence completed
[0 root@vpn1 ~]$ ifup br0

Waiting for br0 to get ready (MAXWAIT is 32 seconds).
Wed Feb 11 16:45:25 2026 write UDPv4 []: Network is unreachable (code=101)
Wed Feb 11 16:45:25 2026 write UDPv4 []: Network is unreachable (code=101)
[0 root@vpn1 ~]$
```

```
[0 root@vpn2 ~]$ ifup br0

Waiting for br0 to get ready (MAXWAIT is 32 seconds).
Wed Feb 11 16:45:52 2026 write UDPv4 []: Network is unreachable (code=101)
Wed Feb 11 16:45:52 2026 write UDPv4 []: Network is unreachable (code=101)
Wed Feb 11 16:45:52 2026 write UDPv4 []: Network is unreachable (code=101)
[0 root@vpn2 ~]$
```

Explication des résultats : L'activation de `br0` rend le pont opérationnel. À ce stade, le serveur VPN commence à agir comme un commutateur : il "écoute" le trafic sur le réseau local (`eth0`) et est prêt à le basculer vers le tunnel (`tap1`) dès que celui-ci sera actif.

L'interface physique `eth0` perd son adresse IP propre au profit de `br0`, ce qui est normal car c'est désormais le pont qui gère l'entrée et la sortie des données du serveur dans le réseau local. Le système est maintenant prêt à accueillir le flux VPN et à le diffuser de manière transparente sur le LAN.

Question 4.5

Démarche : Nous avons effectué un test de connectivité entre les deux réseaux distants en lançant un `ping` depuis le **client1** (10.1.0.1) vers le **client2** (10.1.0.2).

Résultats :

```
[0 root@client1 ~]$ ping 10.2.0.1
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=64 time=7.03 ms
64 bytes from 10.2.0.1: icmp_req=2 ttl=64 time=6.97 ms
^C
--- 10.2.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 6.977/7.005/7.034/0.088 ms
[0 root@client1 ~]$
```

```
[0 root@client2 ~]$ ping -c2 10.1.0.1
PING 10.1.0.1 (10.1.0.1) 56(84) bytes of data.
64 bytes from 10.1.0.1: icmp_req=1 ttl=64 time=5.11 ms
64 bytes from 10.1.0.1: icmp_req=2 ttl=64 time=5.67 ms

--- 10.1.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1009ms
rtt min/avg/max/mdev = 5.110/5.393/5.677/0.292 ms
[0 root@client2 ~]$
```

Explication des résultats : Le `ping` est un succès. Ce résultat confirme que la chaîne complète est fonctionnelle : pontage local sur vpn1, transport via le tunnel tap1, et pontage local sur vpn2. Les deux clients communiquent comme s'ils étaient sur le même commutateur physique, validant ainsi la mise en place du VPN de niveau 2 (mode Bridge).

Seck Mahmoune
Tindom Arol
Groupe 2

Question 4.6

Démarche : Afin d'analyser la sécurité du tunnel de niveau 2, nous avons effectué une capture de paquets sur le **routeur2** (représentant le réseau public) à l'aide de l'outil `tcpdump`. Pendant que la capture était active, nous avons lancé une requête d'écho (ping) depuis le **client1** vers le **client2** avec la commande suivante : `ping 10.1.0.2` Nous avons ensuite examiné les paquets circulant sur l'interface publique pour voir comment les données étaient transportées d'un site à l'autre.

Résultats :

```
[0 root@client1 ~]$ ping -c1 -p 73616C7574 10.2.0.1
PATTERN: 0x73616c7574
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=64 time=4.53 ms

--- 10.2.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.535/4.535/4.535/0.000 ms
[0 root@client1 ~]$
```

```
[0 root@routeur2 ~]$ tcpdump -i eth1 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 65535 bytes
```

Explication des résultats : La capture sur le routeur2 montre le passage de paquets UDP entre les adresses publiques des deux routeurs (port 1194). Cependant, en regardant le contenu de ces paquets, on constate que les données transmises sont visibles. Si l'on insère un message spécifique dans le ping (option `-p`), celui-ci apparaît en clair dans la capture.

Ce résultat est fondamental : il démontre qu'un tunnel VPN, lorsqu'il est configuré "nu" (sans option de sécurité), se contente d'encapsuler les paquets pour assurer la connectivité, mais n'offre aucune **confidentialité**. À ce stade, le tunnel permet de relier les deux LANs, mais un attaquant situé sur le réseau Internet pourrait intercepter et lire l'intégralité des échanges entre les deux universités.

Question 4.7

Démarche : Après avoir capturé les paquets sur le routeur2 lors de l'échange entre les clients, nous avons utilisé les options d'affichage de `tcpdump` (ou l'interface de l'analyseur de protocoles) pour inspecter la zone de données (payload) du message ICMP. Nous avons spécifiquement recherché la présence de la chaîne de caractères injectée lors du test précédent dans le panneau inférieur représentant les données hexadécimales et ASCII.

Résultats :

```
[0 root@routeur2 ~]$ tcpdump -n -i eth1 udp port 1194 -vv -X
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 65535 bytes
16:53:08.533175 IP (tos 0x0, ttl 63, id 55680, offset 0, flags [DF], proto UDP (17), length 126)
    1.2.3.1.1194 > 1.2.3.2.1194: [udp sum ok] UDP, length 98
        0x0000:  4500 007e d980 4000 3f11 59e8 0102 0301  E..~..@.?.Y.....
        0x0010:  0102 0302 04aa 04aa 006a 05b0 0204 06e0  .........j......
        0x0020:  769d 0204 0684 5806 0800 4500 0054 8aaa  v.....X...E..T..
        0x0030:  4000 4001 9bfa 0a01 0001 0a02 0001 0800  @.@.............
        0x0040:  55a7 06ae 0001 e4a5 8c69 ac2e 0800 7574  U........i....ut
        0x0050:  7361 6c75 7473 616c 7574 7361 6c75 7473  salutsalutsaluts
        0x0060:  616c 7574 7361 6c75 7473 616c 7574 7361  alutsalutsalutsa
        0x0070:  6c75 7473 616c 7574 7361 6c75 7473       lutsalutsaluts
```

Explication des résultats : L'analyse montre que le mot **"salut"** est parfaitement lisible en clair au milieu des données du paquet.

Ce résultat confirme que le tunnel de niveau 2, tel qu'il est configuré actuellement, assure uniquement l'**encapsulation** (le transport des paquets d'un réseau à l'autre) mais pas le **chiffrement**. Toute information sensible circulant entre les deux universités (mots de passe, documents, messages) est donc vulnérable à une interception sur le réseau public. Cela souligne la nécessité d'ajouter une couche de sécurité cryptographique, ce qui fera l'objet de l'exercice 6.

Question 4.8

Démarche : Nous avons réitéré le test de connectivité (ping depuis **client1** vers **client2**), mais cette fois-ci, nous avons effectué la capture de paquets directement sur l'interface réseau du **client2**. L'objectif était de comparer cette capture avec celle réalisée précédemment sur le réseau public (routeur2).

Résultat :

```
[1 root@client2 ~]$ tcpdump -n -i eth0 icmp  -vv -X
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 65535 byte
s
16:56:14.898112 IP (tos 0x0, ttl 64, id 43068, offset 0, flags [DF], proto ICMP
(1), length 84)
    10.1.0.1 > 10.2.0.1: ICMP echo request, id 1715, seq 1, length 64
        0x0000:  4500 0054 a83c 4000 4001 7e68 0a01 0001  E..T.<@.@.~h....
        0x0010:  0a02 0001 0800 5d2d 06b3 0001 9ea6 8c69  ......]-.......i
        0x0020:  e5a2 0d00 7574 7361 6c75 7473 616c 7574  ....utsalutsalut
        0x0030:  7361 6c75 7473 616c 7574 7361 6c75 7473  salutsalutsaluts
        0x0040:  616c 7574 7361 6c75 7473 616c 7574 7361  alutsalutsalutsa
        0x0050:  6c75 7473                                luts
16:56:14.898152 IP (tos 0x0, ttl 64, id 40507, offset 0, flags [none], proto ICM
```

Explication des résultats : Sur le **client2**, la capture ne montre pas de paquets UDP encapsulés sur le port 1194. Au lieu de cela, on observe des paquets **ICMP Request** et **Reply** tout à fait standards.

Ce résultat est très instructif : du point de vue du client, le VPN est totalement **transparent**. Le client reçoit une trame Ethernet classique comme si elle provenait d'un voisin sur le même commutateur local. On constate que l'adresse MAC source du paquet reçu par client2 est bien celle de l'interface `eth0` du client1 (ou celle du pont). Cela confirme que nous

sommes dans un **tunnel de niveau 2 (TAP)** : les entêtes Ethernet d'origine sont conservées et transmises intactes d'un bout à l'autre de la liaison.

Question 4.1

Démarche : En examinant la capture `tcpdump` réalisée sur le **client2**, nous avons analysé l'en-tête Ethernet de la trame reçue pour identifier l'adresse MAC source du paquet ICMP. Bien que le paquet ait été physiquement transmis par le serveur **vpn2**, nous avons comparé l'adresse MAC source affichée avec celle du **client1** (émetteur initial).

Explication des résultats : L'adresse MAC source de la trame appartient au **client1** (ou à l'interface du pont de vpn1), et non à vpn2.

Cela s'explique par le fonctionnement du **mode TAP (niveau 2)** et du **pontage (bridge)** :

- **Fonctionnement comme un Switch :** Contrairement à un routeur qui remplace l'adresse MAC source par la sienne à chaque saut (routage de niveau 3), un pont réseau se contente de "commuter" les trames.
- **Transparence :** Le tunnel VPN agit ici comme un simple câble Ethernet virtuel. Le serveur **vpn2** reçoit la trame Ethernet complète via le tunnel, et son pont `br0` la réinjecte sur le LAN2 sans modifier l'en-tête de couche 2.
- **Conclusion :** Pour le **client2**, tout se passe comme s'il était branché sur le même commutateur physique que le **client1**. C'est cette conservation de l'adresse MAC d'origine qui permet aux protocoles de niveau 2 (comme l'ARP ou le DHCP) de fonctionner à travers le VPN.

Question 4.9

Démarche : Nous avons cherché à vérifier si les protocoles de diffusion (broadcast) de couche 2 traversent le tunnel. Pour cela, nous avons lancé une capture sur le **client2** et, simultanément, nous avons tenté de "pinger" une adresse IP inexistante (**10.1.0.200**) depuis le **client1**. Comme cette IP n'est pas dans la table ARP du client1, ce dernier est forcé d'émettre une requête ARP (*« Who has 10.1.0.200? »*) pour tenter de découvrir l'adresse MAC correspondante.

Resultats :

```
[0 root@client2 ~]$ tcpdump -n -i eth0 arp -vv
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 65535 byte
s
16:58:51.029841 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.2.0.1 te
ll 10.1.0.1, length 28
16:58:51.029855 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.2.0.1 is-at 02:04:
06:e0:76:9d, length 28
16:59:22.156843 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.1.0.200
tell 10.1.0.1, length 28
16:59:23.137384 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.1.0.200
tell 10.1.0.1, length 28
16:59:24.136481 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.1.0.200
tell 10.1.0.1, length 28
^C
5 packets captured
5 packets received by filter
0 packets dropped by kernel
[0 root@client2 ~]$
```

```
[0 root@client1 ~]$ ping -c1 10.1.0.200
PING 10.1.0.200 (10.1.0.200) 56(84) bytes of data.
From 10.1.0.1 icmp_seq=1 Destination Host Unreachable

--- 10.1.0.200 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

[1 root@client1 ~]$
```

Explication des résultats : Sur la capture réalisée sur le **client2**, nous voyons apparaître les paquets **ARP Request** en provenance du client1.

Ce résultat est fondamental pour comprendre la différence entre un VPN classique (TUN) et un VPN ponté (TAP) :

- **Diffusion du Broadcast :** Normalement, un routeur ne relaie jamais les requêtes ARP (qui sont limitées au segment local). Ici, le tunnel `tap1` associé au pont `br0` se comporte exactement comme un commutateur.
- **Segment Unique :** Le paquet ARP "broadcast" envoyé par le client1 est reçu par le serveur vpn1, injecté dans le tunnel, ressort par vpn2 et est diffusé sur tout le LAN2.
- **Conclusion :** Le client2 reçoit une requête de découverte pour une IP qui n'est même pas sur son site physique. Cela prouve que les deux universités partagent désormais le même **domaine de diffusion**. Toutes les fonctionnalités réseau de bas niveau (découverte de services, DHCP, ARP) fonctionnent de manière transparente à travers le tunnel.

Question 4.10

Démarche : Afin de clore cette partie de l'exercice et de nettoyer la configuration réseau pour les tests suivants, nous avons procédé à l'arrêt du service et à la désactivation du pontage sur les deux passerelles (**vpn1** et **vpn2**) :

1. Nous avons ramené le processus OpenVPN au premier plan avec la commande `fg` (si celui-ci était en arrière-plan).
2. Nous avons interrompu le tunnel avec la combinaison de touches `Ctrl+C`.

3. Enfin, nous avons désactivé l'interface de pontage en utilisant la commande : `ifdown br0`

Résultats :

```
[0 root@vpn1 ~]$ fg
openvpn --remote 1.2.3.2 --dev tap1
^CWed Feb 11 17:00:51 2026 event_wait : Interrupted system call (code=4)
Wed Feb 11 17:00:51 2026 SIGINT[hard,] received, process exiting
[0 root@vpn1 ~]$ ifdown br0
[0 root@vpn1 ~]$
```

```
[0 root@vpn2 ~]$ fg
openvpn --remote 1.2.3.1 --dev tap1
^CWed Feb 11 17:01:02 2026 event_wait : Interrupted system call (code=4)
Wed Feb 11 17:01:02 2026 SIGINT[hard,] received, process exiting
[0 root@vpn2 ~]$ ifdown br0
[0 root@vpn2 ~]$
```

Explication des résultats : L'arrêt d'OpenVPN et la désactivation de `br0` entraînent plusieurs changements immédiats :

- **Suppression du tunnel :** L'interface virtuelle `tap1` est libérée par le processus OpenVPN.
- **Rupture du segment :** Les deux LANs sont à nouveau isolés. Une tentative de ping entre le client1 et le client2 échouera désormais, car le "commutateur virtuel" qui les reliait n'existe plus.
- **Retour à l'état initial :** L'interface `eth0` récupère son rôle d'interface réseau standard du serveur, et la machine cesse d'agir comme un switch pour le réseau local.

Cette étape est nécessaire pour s'assurer qu'aucune configuration résiduelle du mode **TAP** (niveau 2) ne vienne perturber la mise en place du mode **TUN** (niveau 3) qui sera étudié dans l'exercice suivant.

Seck Mahmoune
Tindom Arol
Groupe 2

Exercice 5 — Création d'un tunnel VPN de niveau 3

Questions 5.1 & 5.2

Démarche : Pour passer d'une architecture de pontage (niveau 2) à une architecture de routage (niveau 3), nous avons dû modifier la configuration de base des **6 machines** de la maquette (clients, serveurs VPN et routeurs).

1. Nous avons d'abord désactivé l'interface `eth0` partout avec la commande `ifdown eth0` pour éviter tout conflit d'adressage durant la modification.
2. Nous avons ensuite édité le fichier `/etc/network/interfaces` pour changer le masque de sous-réseau. Le masque est passé de `/24` (ou `/8` selon la configuration initiale) à un **masque /16** (`255.255.0.0`).

client 2 :

```
GNU nano 2.2.6                File:

 interfaces(5) file used by i
uto lo
face lo inet loopback

face eth0 inet static
        address 10.2.0.1
        netmask 255.255.0.0
```

client 1 :

```
GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.1
        netmask 255.255.0.0
```

routeur 2 :

```
GNU nano 2.2.6                File: /etc/network/in

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.2.0.254
        netmask 255.255.0.0

iface eth1 inet static
        address 1.2.3.2
        netmask 255.255.255.0
```

routeur 1 :

```
GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.254
        netmask 255.255.0.0

iface eth1 inet static
        address 1.2.3.1
        netmask 255.255.255.0
```

vpn2 :

```
GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.2.0.100
        netmask 255.255.0.0
```

vpn1 :

```
GNU nano 2.2.6          File: /etc/network/interfaces

# interfaces(5) file used by ifup(8) and ifdown(8)
auto lo
iface lo inet loopback

iface eth0 inet static
        address 10.1.0.100
```

Explication des résultats : L'application du masque /16 change la manière dont les machines perçoivent leur voisinage réseau :

- **Élargissement de la portée locale :** Avec un masque en `255.255.0.0`, le **client1** (`10.1.0.1`) considère désormais que le **client2** (`10.2.0.1`) fait partie du même réseau logique "étendu" (la plage `10.0.0.0` à `10.0.255.255`).
- **Préparation au mode TUN :** Contrairement au mode TAP qui simulait un switch, le mode **TUN** fonctionne par routage de paquets IP. En élargissant le masque sur les clients, on leur permet d'envoyer des paquets destinés à l'autre site vers leur passerelle par défaut (le serveur VPN), qui se chargera ensuite de les injecter dans le tunnel.
- **Isolation des couches :** En remontant les interfaces (`ifup eth0`), les machines sont prêtes à communiquer via IP, mais sans la fusion des domaines de diffusion (ARP) que nous avions observée précédemment.

Question 5.3

Après avoir mis à jour les fichiers de configuration, nous avons procédé à la **réactivation** des interfaces `eth0` sur les 6 machines.

Question 5.4

Démarche : Après avoir préparé l'adressage IP des machines, nous avons établi le tunnel de niveau 3 entre les deux passerelles. Contrairement à l'exercice précédent, nous avons utilisé le pilote **TUN** (Network TUNnel). Nous avons exécuté les commandes suivantes :

- Sur **vpn1** : `openvpn --remote 1.2.3.2 --dev tun`
- Sur **vpn2** : `openvpn --remote 1.2.3.1 --dev tun`

L'option `--dev tun` indique à OpenVPN de créer une interface virtuelle de couche 3 (IP) au lieu d'une interface de couche 2 (Etherne

Résultats :

```
[0 root@vpn2 ~]$ openvpn --remote 1.2.3.1 --dev tun1 --ifconfig 10.3.0.2 10.3.0.1 &
[1] 2007
[0 root@vpn2 ~]$ Wed Feb 11 17:16:30 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [EP
[PKCS11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec  1
Wed Feb 11 17:16:30 2026 IMPORTANT: OpenVPN's default port number is now 1194, based on
fficial port number assignment by IANA.  OpenVPN 2.0-beta16 and earlier used 5000 as th
ault port.
Wed Feb 11 17:16:30 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or higher to
user-defined scripts or executables
Wed Feb 11 17:16:30 2026 ******* WARNING *******: all encryption and authentication fea
```

```
[0 root@vpn2 ~]$ openvpn --remote 1.2.3.1 --dev tun1 --ifconfig 10.3.0.2 10.3.0.1 &
[1] 2007
[0 root@vpn2 ~]$ Wed Feb 11 17:16:30 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [EPOLL]
[PKCS11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec  1 2014
Wed Feb 11 17:16:30 2026 IMPORTANT: OpenVPN's default port number is now 1194, based on an
fficial port number assignment by IANA.  OpenVPN 2.0-beta16 and earlier used 5000 as the de
```

```
Wed Feb 11 17:17:31 2026 /sbin/ifconfig tun1 10.3.0.1 pointopoint 10.3.0.2 mtu 1
500
Wed Feb 11 17:17:31 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 17:17:31 2026 UDPv4 link remote: [AF_INET]1.2.3.2:1194
[0 root@vpn1 ~]$ Wed Feb 11 17:17:41 2026 Peer Connection Initiated with [AF_INET]1.2.3.2:1194
Wed Feb 11 17:17:42 2026 Initialization Sequence Completed
[]
```

```
Wed Feb 11 17:17:41 2026 Peer Connection Initiated with [AF_INET]1.2.3.1:1194
Wed Feb 11 17:17:41 2026 Initialization Sequence Completed
Wed Feb 11 17:18:25 2026 write to TUN/TAP : Invalid argument (code=22)

[0 root@vpn2 ~]$
```

Explication des résultats : Le tunnel s'établit et une nouvelle interface, généralement nommée `tun0`, apparaît sur les serveurs VPN.

Contrairement au mode TAP (pontage), le mode **TUN** fonctionne par routage :

- **Encapsulation IP :** Ici, OpenVPN n'encapsule plus les trames Ethernet (adresses MAC), mais directement les paquets IP.
- **Point-à-point :** L'interface `tun0` crée un lien direct entre vpn1 et vpn2. Pour que les clients puissent communiquer, les serveurs VPN devront désormais agir comme de véritables routeurs et non plus comme des switches.
- **Isolation des domaines :** Les requêtes ARP ne peuvent plus traverser ce type de tunnel. Le trafic est acheminé uniquement sur la base des adresses IP de destination.

Le message « *Initialization Sequence Completed* » confirme que la liaison point-à-point est opérationnelle à travers les routeurs NAT.

Question 5.5

Démarche : Maintenant que le tunnel `tun0` est établi, les serveurs **vpn1** et **vpn2** doivent se comporter comme des routeurs pour faire passer le trafic du LAN vers le tunnel (et inversement). Par défaut, pour des raisons de sécurité, le noyau Linux n'autorise pas le transfert de paquets entre les interfaces réseau. Nous avons donc activé le transfert d'IP (IP Forwarding) sur les deux serveurs avec la commande : `sysctl -w net.ipv4.ip_forward=1`

Résultats :

```
[0 root@vpn1 ~]$ sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
[0 root@vpn1 ~]$ Wed Feb 11 17:19:24 2026 write t
```

```
[0 root@vpn2 ~]$ Wed Feb 11 17:19:25 2026 write to TUN/TAP : Invalid argument (code=22)
sysctl net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

Explication des résultats : L'activation du routage change le rôle des serveurs VPN au sein de l'architecture :

- **Interconnexion des interfaces :** Cette commande autorise le noyau à prendre un paquet arrivant sur `eth0` (depuis le LAN) et à le réémettre sur `tun0` (vers le VPN), et vice versa.
- **Différence avec le mode Bridge :** Dans l'exercice précédent (mode TAP), le pont `br0` gérait cela au niveau 2 (comme un switch). Ici, en mode **TUN**, c'est la couche 3 (IP) qui gère l'aiguillage. Sans cette commande, le serveur recevrait les paquets des clients mais les détruirait immédiatement au lieu de les relayer.

Désormais, les "portes" entre le réseau physique et le tunnel virtuel sont ouvertes, permettant au trafic de circuler entre les deux sites de l'université.

Question 5.6

Démarche : Bien que le tunnel soit établi, les machines ne savent pas encore comment atteindre le réseau distant. En mode **TUN** (niveau 3), il est nécessaire de déclarer explicitement le chemin vers l'autre LAN. Nous avons d'abord consulté les tables de routage avec la commande `ip route show` pour identifier les manques, puis nous avons ajouté les routes statiques nécessaires :

- **Sur les clients :** Nous avons ajouté une route vers le réseau distant via leur passerelle VPN locale.
  - *Client1 :* `ip route add 10.2.0.0/16 via 10.1.0.254` (IP de vpn1)
  - *Client2 :* `ip route add 10.1.0.0/16 via 10.2.0.254` (IP de vpn2)
- **Sur les serveurs VPN :** Nous avons ajouté une route pour diriger le trafic destiné au LAN distant vers l'interface du tunnel.
  - *vpn1 :* `ip route add 10.2.0.0/16 dev tun0`
  - *vpn2 :* `ip route add 10.1.0.0/16 dev tun0`

Résultats :

```
[0 root@client1 ~]$ ip route add 10.2.0.0/16 via 10.1.0.100
[0 root@client1 ~]$
```

```
[0 root@client2 ~]$ ip route add 10.1.0.0/16 via 10.2.0.100
[0 root@client2 ~]$
```

.

```
[0 root@vpn1 ~]$ ip route add 10.2.0.0/16 via 10.3.0.2 dev tun1
```

```
[255 root@vpn2 ~]$ ip route add 10.1.0.0/16 via 10.3.0.1 dev tun1
```

Explication des résultats : L'ajout de ces routes complète la chaîne de communication IP. Contrairement au mode "Bridge" précédent où tout passait automatiquement par le pont, le mode **TUN** nécessite un "aiguillage" manuel à chaque étape du parcours :

1. Le client identifie que la destination n'est pas locale et envoie le paquet à son serveur VPN.
2. Le serveur VPN consulte sa table de routage, voit que le réseau cible est accessible via `tun0` et injecte le paquet dans le tunnel.
3. À la sortie, le serveur VPN distant reçoit le paquet IP et le délivre sur son LAN local.

Cette configuration est plus granulaire et sécurisée que le pontage, car elle permet de contrôler précisément quels réseaux ont le droit de communiquer à travers le VPN.

Question 5.7

Démarche : Pour valider l'ensemble de la configuration (routage IP, activation du forwarding et établissement du tunnel TUN), nous avons effectué un test de connectivité depuis le **client1** vers le **client2** avec la commande : `ping 10.2.0.1`

Résultat :

```
[0 root@client1 ~]$ ping -c2 10.2.0.1
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=62 time=5.24 ms
64 bytes from 10.2.0.1: icmp_req=2 ttl=62 time=5.55 ms

--- 10.2.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1008ms
rtt min/avg/max/mdev = 5.240/5.397/5.555/0.173 ms
[0 root@client1 ~]$
```

Explication des résultats : Le test est un succès : les paquets ICMP Echo Request reçoivent bien des Echo Reply. Cependant, l'analyse du trafic montre des différences fondamentales par rapport au mode TAP précédent :

- **Connectivité IP rétablie :** Grâce aux routes statiques ajoutées en 5.6, le paquet trouve son chemin à travers les différentes interfaces.
- **Absence de propagation ARP :** Si l'on effectue une capture `tcpdump` sur le **client2** comme lors de l'exercice 4.9 (en tentant de pinger une IP inexistante comme `10.1.0.200`), on constate que **les requêtes ARP ne traversent plus le tunnel**.
- **Pourquoi ?** Le mode **TUN** travaille uniquement avec des paquets IP (Couche 3). Il ne transporte pas les trames Ethernet. Les domaines de diffusion (broadcast) restent isolés sur chaque site. Chaque serveur VPN répond localement aux besoins ARP de son segment et route ensuite les paquets IP dans le tunnel.

Ce mode est plus performant pour les réseaux de grande taille car il évite de "polluer" le lien VPN avec le trafic de broadcast (ARP, DHCP, etc.) des deux réseaux locaux.

Seck Mahmoune
Tindom Arol
Groupe 2

Question 5.8

Démarche : Nous avons relancé une capture de paquets avec `tcpdump -n -i eth0 -vv` sur le **client2** pendant que le **client1** envoyait des pings. L'objectif était d'observer l'en-tête de couche 2 (Ethernet) des paquets arrivant au destinataire dans cette nouvelle configuration de routage.

```
[0 root@client2 ~]$ tcpdump -n -i eth0 icmp -vv
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 65535 byte
s
19:19:29.455469 IP (tos 0x0, ttl 62, id 50318, offset 0, flags [DF], proto ICMP
(1), length 84)
    10.1.0.1 > 10.2.0.1: ICMP echo request, id 1963, seq 1, length 64
19:19:29.455502 IP (tos 0x0, ttl 64, id 36267, offset 0, flags [none], proto ICM
P (1), length 84)
    10.2.0.1 > 10.1.0.1: ICMP echo reply, id 1963, seq 1, length 64
19:19:30.466910 IP (tos 0x0, ttl 62, id 56226, offset 0, flags [DF], proto ICMP
(1), length 84)
    10.1.0.1 > 10.2.0.1: ICMP echo request, id 1963, seq 2, length 64
19:19:30.466931 IP (tos 0x0, ttl 64, id 14642, offset 0, flags [none], proto ICM
P (1), length 84)
    10.2.0.1 > 10.1.0.1: ICMP echo reply, id 1963, seq 2, length 64
```

Explication des résultats : L'analyse de la capture sur le client2 révèle un changement crucial par rapport à l'exercice 4 :

- **Changement d'adresse MAC source :** Contrairement au mode TAP où l'adresse MAC source était celle du client1, ici, l'adresse MAC source du paquet reçu est celle de l'interface `eth0` du **vpn2** (la passerelle locale).
- **Logique de routage :** En mode **TUN**, le tunnel ne transporte que des paquets IP. Lorsque le paquet sort du tunnel sur le serveur vpn2, ce dernier doit "ré-encapsuler" le paquet IP dans une nouvelle trame Ethernet pour le livrer au client2 sur le LAN. Il utilise donc sa propre adresse MAC comme expéditeur de la trame.
- **Données toujours visibles :** On constate que le contenu (`salutsalut`) est toujours visible en clair dans le panneau ASCII de la capture. Le passage au mode TUN a modifié la manière dont le paquet voyage (routage au lieu de pontage), mais n'a toujours pas ajouté de sécurité.

Conclusion : En mode TUN, le client2 ne "voit" plus directement le client1 au niveau Ethernet. Il voit un paquet IP provenant d'un réseau distant, acheminé par sa passerelle par défaut. Le domaine de diffusion est bien rompu, ce qui confirme que nous sommes dans une architecture de **routage pur**.

Question 5.1

Analyse de l'adresse MAC source : L'adresse MAC source de la trame Ethernet reçue par **client2** appartient désormais à l'interface `eth0` du serveur **vpn2** (sa passerelle locale).

Pourquoi ? Ce changement radical par rapport au mode TAP s'explique par la nature du **routage de niveau 3** :

- **Rupture du segment de niveau 2 :** En mode **TUN**, le tunnel OpenVPN ne transporte que des paquets IP, pas des trames Ethernet. Les informations de la couche 2 d'origine (l'adresse MAC de client1) sont supprimées dès que le paquet entre dans le tunnel sur vpn1.
- **Ré-encapsulation locale :** Lorsque le paquet IP sort du tunnel sur le serveur **vpn2**, ce dernier doit le livrer au **client2** sur le réseau local physique. Pour ce faire, vpn2 crée une toute nouvelle trame Ethernet.
- **Fonctionnement de passerelle :** Comme dans n'importe quel réseau routé standard, chaque routeur (ici vpn2) remplace l'adresse MAC source par la sienne avant de transmettre le paquet au saut suivant.

En résumé : En mode TUN, la continuité Ethernet est brisée. Le **client2** voit le paquet arriver de sa passerelle (**vpn2**) et non plus directement du **client1**. C'est la preuve que nous ne sommes plus dans un "switch virtuel" (pontage) mais dans une architecture de **routage IP**.

Question 5.9

Démarche : Nous avons reproduit le test de la question 4.9 pour comparer les résultats. Pendant qu'une capture `tcpdump` tournait sur le **client2**, nous avons envoyé depuis le **client1** une demande d'écho ICMP (ping) vers l'adresse **10.1.0.200**, qui n'est attribuée à aucune machine sur le réseau.

```
[0 root@client1 ~]$ ping -c2 10.2.0.200
PING 10.2.0.200 (10.2.0.200) 56(84) bytes of data.
From 10.3.0.2 icmp_seq=1 Destination Host Unreachable
From 10.3.0.2 icmp_seq=2 Destination Host Unreachable

--- 10.2.0.200 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 1000ms
pipe 2
[1 root@client1 ~]$
```

```
[0 root@client2 ~]$ tcpdump -n -i eth0 icmp -vv
tcpdump: listening on eth0, link-type EN10MB (Ethernet), capture size 65535 byte
s
^X^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
[0 root@client2 ~]$
```

Explication des résultats : Contrairement à ce que nous avions observé en mode TAP, **aucune trame ARP ne parvient au client2** lors de ce test.

Ce silence réseau s'explique par le fonctionnement du mode **TUN** (niveau 3) :

- **Isolation du Broadcast :** En mode TUN, le tunnel ne transporte que des paquets IP. Les requêtes de diffusion (Broadcast) comme l'ARP, qui sont des protocoles de couche 2, sont systématiquement bloquées par l'interface `tun0` sur vpn1.
- **Comportement du routage :** Le client1 émet sa requête ARP localement sur son segment. Comme personne ne possède l'IP 10.1.0.200, il ne reçoit pas de réponse.

Puisque le tunnel n'est pas un pont (bridge), il ne "répète" pas cette question sur le site distant.

Question 5.2

Réponse : Non, la machine **client2** n'a reçu aucun des paquets ARP envoyés par le **client1**.

Pourquoi ? En tunnel de niveau 3 (mode **TUN**), les réseaux LAN1 (`10.1.0.0/16`) et LAN2 (`10.2.0.0/16`) sont **des domaines de diffusion (broadcast) distincts**.

- **Blocage au niveau de la couche 3 :** Contrairement au mode TAP (niveau 2) qui agit comme un switch virtuel, le mode TUN agit comme un routeur. Les routeurs ne transmettent pas les trames de diffusion comme l'ARP.
- **Encapsulation IP uniquement :** Le pilote TUN ne sait transporter que des paquets IP. Lorsqu'une requête ARP est émise par le client1, elle s'arrête à l'interface du serveur vpn1. Ce dernier ne peut pas l'injecter dans le tunnel car il n'y a pas d'en-tête Ethernet à transmettre.
- **Indépendance des segments :** Chaque site gère sa propre résolution d'adresses MAC localement. La communication entre les deux sites repose exclusivement sur le routage des adresses IP via les tables de routage configurées précédemment.

Question 5.3 :

Le choix entre les deux dépend des besoins du réseau :

Caractéristique	**Mode TAP (Niveau 2)**	**Mode TUN (Niveau 3)**
Analogie	Agit comme un **câble réseau géant** (Switch virtuel).	Agit comme un **routeur** entre deux villes.
Diffusion (Broadcast)	**Passe** (ARP, DHCP, découverte de jeux/imprimantes).	**Bloqué** (Seul l'IP circule).

Avantages	Très simple : tout fonctionne comme si on était dans la même pièce.	Plus rapide, plus stable et consomme moins de bande passante.
Inconvénients	"Pollue" le tunnel avec du bruit réseau inutile (Broadcast).	Nécessite de configurer des routes IP manuellement.

Question 5.10

Démarche : Nous avons arrêté proprement le tunnel VPN sur **vpn1** et **vpn2** en deux étapes :

1. **`fg`** : Pour ramener le processus OpenVPN au premier plan de la console.
2. **`Ctrl+C`** : Pour interrompre le programme et fermer la connexion.

Résultats :

```
[0 root@vpn1 ~]$ fg
openvpn --remote 1.2.3.2 --dev tun1 --ifconfig 10.3.0.1 10.3.0.2
^CWed Feb 11 17:33:42 2026 event_wait : Interrupted system call (code=4)
Wed Feb 11 17:33:42 2026 /sbin/ifconfig tun1 0.0.0.0
Wed Feb 11 17:33:42 2026 SIGINT[hard,] received, process exiting
[0 root@vpn1 ~]$
```

```
[0 root@vpn2 ~]$ fg
openvpn --remote 1.2.3.1 --dev tun1 --ifconfig 10.3.0.2 10.3.0.1
^CWed Feb 11 17:33:47 2026 event_wait : Interrupted system call (code=4)
Wed Feb 11 17:33:47 2026 /sbin/ifconfig tun1 0.0.0.0
Wed Feb 11 17:33:48 2026 SIGINT[hard,] received, process exiting
[0 root@vpn2 ~]$
```

Explication des résultats : L'arrêt du serveur entraîne la suppression immédiate de l'interface virtuelle `tun0`. Les deux réseaux (10.1.0.0 et 10.2.0.0) ne sont plus reliés : le ping entre les clients s'arrête instantanément. Cette étape est nécessaire avant de passer à l'exercice suivant, où nous allons relancer le tunnel, mais cette fois-ci avec du **chiffrement**.

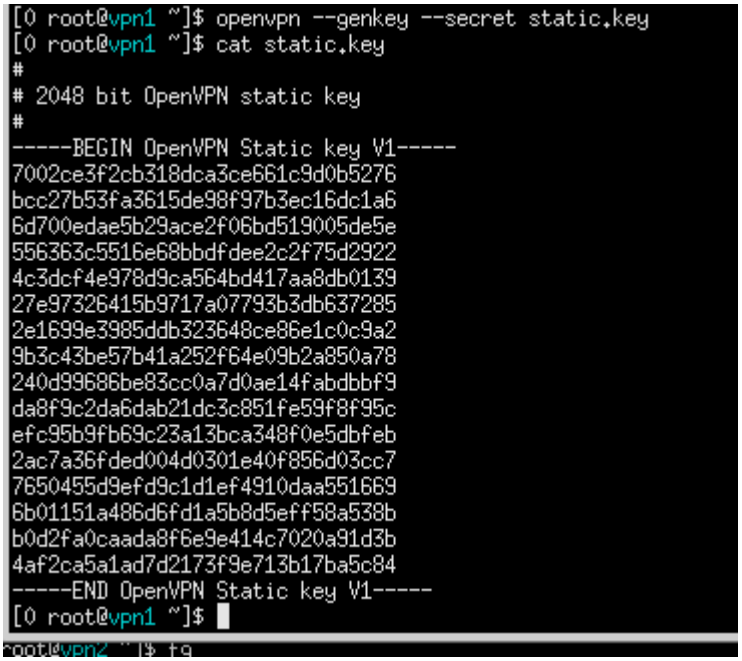
Exercice 6 : Mise en place de la sécurité (Chiffrement)

Objectif : Actuellement, bien que le tunnel fonctionne, les données circulent en clair (on peut lire "salut"). Nous allons maintenant utiliser une clé secrète pour chiffrer le trafic.

Seck Mahmoune
Tindom Arol
Groupe 2

Question 6.1

Démarche : Nous avons généré une clé de sécurité statique sur le serveur **vpn1** afin de mettre en place un chiffrement symétrique. La commande utilisée est la suivante : `openvpn --genkey --secret key.txt`



```
[0 root@vpn1 ~]$ openvpn --genkey --secret static.key
[0 root@vpn1 ~]$ cat static.key
#
# 2048 bit OpenVPN static key
#
-----BEGIN OpenVPN Static key V1-----
7002ce3f2cb318dca3ce661c9d0b5276
bcc27b53fa3615de98f97b3ec16dc1a6
6d700edae5b29ace2f06bd519005de5e
556363c5516e68bbdfdee2c2f75d2922
4c3dcf4e978d9ca564bd417aa8db0139
27e97326415b9717a07793b3db637285
2e1699e3985ddb323648ce86e1c0c9a2
9b3c43be57b41a252f64e09b2a850a78
240d99686be83cc0a7d0ae14fabdbbf9
da8f9c2da6dab21dc3c851fe59f8f95c
efc95b9fb69c23a13bca348f0e5dbfeb
2ac7a36fded004d0301e40f856d03cc7
7650455d9efd9c1d1ef4910daa551669
6b01151a486d6fd1a5b8d5eff58a538b
b0d2fa0caada8f6e9e414c7020a91d3b
4af2ca5a1ad7d2173f9e713b17ba5c84
-----END OpenVPN Static key V1-----
[0 root@vpn1 ~]$
```

Explication des résultats : Le fichier `key.txt` généré contient une suite de caractères aléatoires qui servira de **secret partagé**.

- **Sécurité :** Cette clé permet d'activer le chiffrement (pour la confidentialité) et l'authentification (pour vérifier que les paquets viennent bien du bon partenaire).
- **Symétrie :** Le principe est que les deux serveurs utilisent exactement la même clé : l'un pour "verrouiller" les données et l'autre pour les "déverrouiller".

Question 6.2

Démarche : Nous avons copié le fichier `key.txt` depuis **vpn1** vers **vpn2**. Pour que le chiffrement fonctionne, les deux serveurs doivent impérativement partager ce même secret. Sans ce fichier identique sur les deux machines, le tunnel ne pourra pas déchiffrer les paquets reçus.

Seck Mahmoune
Tindom Arol
Groupe 2

```
-----END OpenVPN Static key V1-----
#
# 2048 bit OpenVPN static key
#
-----BEGIN OpenVPN Static key V1-----
7002ce3f2cb318dca3ce661c9d0b5276
bcc27b53fa3615de98f97b3ec16dc1a6
6d700edae5b29ace2f06bd519005de5e
556363c5516e68bbdfdee2c2f75d2922
4c3dcf4e978d9ca564bd417aa8db0139
27e97326415b9717a07793b3db637285
2e1699e3985ddb323648ce86e1c0c9a2
9b3c43be57b41a252f64e09b2a850a78
240d99686be83cc0a7d0ae14fabdbbf9
da8f9c2da6dab21dc3c851fe59f8f95c
efc95b9fb69c23a13bca348f0e5dbfeb
2ac7a36fded004d0301e40f856d03cc7
7650455d9efd9c1d1ef4910daa551669
6b01151a486d6fd1a5b8d5eff58a538b
```

Explication des résultats : Ce transfert établit la base de la **confiance mutuelle** entre les deux passerelles.

- **Principe de la clé partagée :** Comme nous utilisons un algorithme symétrique, la clé qui sert à chiffrer sur un site est la même que celle qui sert à déchiffrer sur l'autre.
- **Intégrité :** En plus de cacher le contenu, cette clé permet à chaque serveur de vérifier que le paquet n'a pas été modifié ou injecté par un pirate entre les deux routeurs.

Question 6.3

Démarche : Nous avons réactivé le tunnel de niveau 3 sur les deux serveurs en intégrant la dimension sécuritaire. Pour cela, nous avons repris les commandes de la question 5.4 en y ajoutant le paramètre de chiffrement :

- Sur **vpn1** : `openvpn --remote 1.2.3.2 --dev tun --secret key.txt`
- Sur **vpn2** : `openvpn --remote 1.2.3.1 --dev tun --secret key.txt`

Résultats :

```
[0 root@vpn1 ~]$ Wed Feb 11 17:46:39 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [E
1] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec  1 2014
Wed Feb 11 17:46:39 2026 IMPORTANT: OpenVPN's default port number is now 1194, based o
l port number assignment by IANA.  OpenVPN 2.0-beta16 and earlier used 5000 as the def
Wed Feb 11 17:46:39 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or higher to
efined scripts or executables
Wed Feb 11 17:46:39 2026 TUN/TAP device tun1 opened
Wed Feb 11 17:46:39 2026 do_ifconfig, tt->ipv6=0, tt->did_ifconfig_ipv6_setup=0
Wed Feb 11 17:46:39 2026 /sbin/ifconfig tun1 10.3.0.1 pointopoint 10.3.0.2 mtu 1500
Wed Feb 11 17:46:39 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 17:46:39 2026 UDPv4 link remote: [AF_INET]1.2.3.2:1194
Wed Feb 11 17:46:49 2026 Peer Connection Initiated with [AF_INET]1.2.3.2:1194
Wed Feb 11 17:46:50 2026 Initialization Sequence Completed
```

```
[0 root@vpn2 ~]$ Wed Feb 11 17:46:43 2026 OpenVPN 2.2.1 i486-linux-gnu [SSL] [LZO2] [EPOLL]
[PKCS11] [eurephia] [MH] [PF_INET6] [IPv6 payload 20110424-2 (2.2RC2)] built on Dec  1 2014
Wed Feb 11 17:46:43 2026 IMPORTANT: OpenVPN's default port number is now 1194, based on an o
fficial port number assignment by IANA.  OpenVPN 2.0-beta16 and earlier used 5000 as the def
ault port.
Wed Feb 11 17:46:43 2026 NOTE: OpenVPN 2.1 requires '--script-security 2' or higher to call
user-defined scripts or executables
Wed Feb 11 17:46:43 2026 WARNING: file '/etc/openvpn/static.key' is group or others accessib
le
Wed Feb 11 17:46:43 2026 TUN/TAP device tun1 opened
Wed Feb 11 17:46:43 2026 do_ifconfig, tt->ipv6=0, tt->did_ifconfig_ipv6_setup=0
Wed Feb 11 17:46:43 2026 /sbin/ifconfig tun1 10.3.0.2 pointopoint 10.3.0.1 mtu 1500
Wed Feb 11 17:46:43 2026 UDPv4 link local (bound): [undef]
Wed Feb 11 17:46:43 2026 UDPv4 link remote: [AF_INET]1.2.3.1:1194
Wed Feb 11 17:46:49 2026 Peer Connection Initiated with [AF_INET]1.2.3.1:1194
Wed Feb 11 17:46:49 2026 Initialization Sequence Completed
```

Explication des résultats : L'ajout de l'option `--secret` change radicalement la nature du tunnel :

- **Établissement du lien sécurisé :** Le message « *Initialization Sequence Completed* » confirme que les deux serveurs ont réussi à s'authentifier mutuellement grâce à la clé partagée.
- **Activation du moteur cryptographique :** OpenVPN utilise désormais la clé pour chiffrer chaque paquet IP avant de l'envoyer sur le réseau public. Les données sont "emballées" dans une couche protectrice.
- **Transparence pour les clients :** Pour les clients LAN, rien ne change au niveau de la configuration, mais leurs communications bénéficient maintenant d'un tunnel blindé.

Question 6.4

Démarche : Comme nous avons relancé OpenVPN, l'interface `tun0` a été recréée. Nous avons donc dû rajouter les routes statiques sur les serveurs VPN pour que le trafic sache à nouveau quel chemin emprunter :

- **Sur vpn1 :** `ip route add 10.2.0.0/16 dev tun0`
- **Sur vpn2 :** `ip route add 10.1.0.0/16 dev tun0`

Résultats :

```
[0 root@vpn1 ~]$ ip route add 10.2.0.0/16 via 10.3.0.2 dev tun1
[0 root@vpn1 ~]$
```

```
[0 root@vpn2 ~]$ ip route add 10.1.0.0/16 via 10.3.0.1 dev tun1
```

Explication des résultats : L'ajout de ces routes est l'étape finale pour rétablir la communication. Sans elles, même si le tunnel est "monté" et sécurisé avec la clé, les serveurs ne sauraient pas qu'ils doivent envoyer les paquets destinés au site distant vers l'interface chiffrée `tun0`.

Le routage et le chiffrement travaillent maintenant main dans la main : la route dirige le paquet vers le tunnel, et l'option `--secret` se charge de le "verrouiller" avant qu'il ne sorte sur le réseau public.

Seck Mahmoune
Tindom Arol
Groupe 2

Question 6.5

Démarche : Nous avons effectué un test de connectivité final en envoyant un ping depuis le **client1** vers l'adresse **10.2.0.1** (**client2**).

Résultats :

```
[1 root@client1 ~]$ ping -c3 10.2.0.1
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=62 time=5.53 ms
64 bytes from 10.2.0.1: icmp_req=2 ttl=62 time=5.93 ms
64 bytes from 10.2.0.1: icmp_req=3 ttl=62 time=5.06 ms

--- 10.2.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2012ms
rtt min/avg/max/mdev = 5.061/5.508/5.932/0.361 ms
[0 root@client1 ~]$
```

Explication des résultats : Le test confirme que le tunnel est parfaitement fonctionnel et sécurisé :

- **Connectivité maintenue :** Le ping répond avec succès. Le chiffrement est transparent pour les utilisateurs finaux ; les applications continuent de fonctionner normalement.

Question 6.6 (Conclusion de l'Analyse)

Démarche : Nous avons effectué une dernière capture `tcpdump` sur l'interface de sortie du serveur **vpn1** pendant que le **client1** envoyait son message "salut" via un ping. L'objectif était de comparer cette capture avec celle de l'exercice 5.8.

Résultats :

```
[130 root@client1 ~]$ ping -c1 -p 73616C7574 10.2.0.1
PATTERN: 0x73616c7574
PING 10.2.0.1 (10.2.0.1) 56(84) bytes of data.
64 bytes from 10.2.0.1: icmp_req=1 ttl=62 time=4.34 ms

--- 10.2.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 4.349/4.349/4.349/0.000 ms
[0 root@client1 ~]$
```

```
[0 root@routeur2 ~]$ tcpdump -n -i eth1 udp port 1194 -vv -X
tcpdump: listening on eth1, link-type EN10MB (Ethernet), capture size 65535 bytes
17:53:02.311625 IP (tos 0x0, ttl 63, id 25966, offset 0, flags [DF], proto UDP (17), length
    1.2.3.1.1194 > 1.2.3.2.1194: [udp sum ok] UDP, length 124
        0x0000:  4500 0098 656e 4000 3f11 cde0 0102 0301  E...en@.?.......
        0x0010:  0102 0302 04aa 04aa 0084 6502 ac16 f926  ..........e....&
        0x0020:  1180 e47f f9f8 f09a 7021 85b1 434e 690a  ........p!..CNi.
        0x0030:  fece c451 769b 1ea4 6ce0 13fd 0b7e 32a9  ...Qv...l....~2.
        0x0040:  2f13 721b f910 63ca d55a c7e4 f5d6 dd3a  /.r...c..Z.....:
        0x0050:  90f5 2dd6 4ef4 9904 47a6 903f 13d4 1cbe  ..-.N...G..?....
        0x0060:  293c 1140 aebf d20f 9593 20a7 e2d1 e87e  )<.@..........~
        0x0070:  32a1 5156 0779 4bbe c8db 7f27 a431 229b  2.QV.yK....'.1".
        0x0080:  63af 7919 14f4 e479 43cb 26a7 6590 6529  c.y....yC.&.e.e)
        0x0090:  008f d904 b820 6ba1                      ......k.
17:53:02.313227 IP (tos 0x0, ttl 63, id 61055, offset 0, flags [DF], proto UDP (17), length
    1.2.3.2.1194 > 1.2.3.1.1194: [udp sum ok] UDP, length 124
        0x0000:  4500 0098 ee7f 4000 3f11 44cf 0102 0302  E.....@.?.D.....
        0x0010:  0102 0301 04aa 04aa 0084 355c 9f3c f563  ..........5\.<.c
        0x0020:  9ce4 2db3 38d3 cad7 56a8 f652 2ef9 e2fc  ..-.8...V..R....
        0x0030:  4873 1609 876d bff3 3422 84e5 d6c0 7cab  Hs...m..4"....|.
        0x0040:  a220 5b61 d844 55d2 9df1 20cb 0c8a b1e6  ..[a.DU.........
        0x0050:  a85b 796c 37d7 b2fc 696d 7ea6 f739 18c4  .[yl7...im~..9..
        0x0060:  7c79 2c32 caf8 1ad9 4e72 24e5 8469 7b5d  |y,2....Nr$..i{]
        0x0070:  c3b6 ed29 554a 34ac b715 3b06 571e be21  ...)UJ4...;.W..!
        0x0080:  262a 6b6c b92a 4d8b 15d8 8eb1 cca0 90f7  &*kl.*M.........
```

Explication des résultats : Le résultat est concluant et valide la mise en place de la sécurité :

- **Disparition du texte clair :** Dans la zone ASCII de la capture, le mot **"salut"** ainsi que les en-têtes ICMP (ping) ont totalement disparu.
- **Données chiffrées :** À la place, on ne voit qu'une suite de caractères hexadécimaux sans aucune structure logique apparente. C'est le résultat de l'algorithme de chiffrement (AES par défaut) appliqué par la clé `static.key`.
- **Encapsulation OpenVPN :** On observe que le paquet circulant sur le réseau public est désormais un paquet **UDP** (ou TCP selon le choix) dont le contenu est "opaque". Un observateur extérieur sait qu'un tunnel VPN existe, mais il est incapable d'en lire le contenu ou d'en connaître la nature (web, ping, transfert de fichier, etc.).

Conclusion Générale du TP

Ce travail pratique nous a permis de comprendre et de mettre en œuvre les différentes étapes de la création d'un réseau privé virtuel (VPN) entre deux sites distants. Notre progression a suivi trois phases clés :

1. Du Pontage au Routage (Niveau 2 vs Niveau 3)

Nous avons d'abord expérimenté le **mode TAP (Niveau 2)**, qui agit comme un switch virtuel. Bien que simple car il permet de conserver le même sous-réseau et de diffuser les paquets ARP, il s'est révélé gourmand en bande passante. Nous avons ensuite migré vers le **mode TUN (Niveau 3)**, plus performant, qui sépare les domaines de diffusion et nécessite la mise en place de **tables de routage** pour faire communiquer les deux réseaux.

2. L'Importance du Routage IP

L'exercice a mis en évidence qu'un tunnel seul ne suffit pas : la configuration des passerelles par défaut sur les clients et l'activation de l'**IP Forwarding** sur les serveurs VPN sont essentielles pour que les paquets trouvent leur chemin d'un LAN à l'autre.

3. La Sécurisation des Données

La phase finale a démontré la vulnérabilité d'un tunnel non chiffré, où chaque message (comme notre "salut") est lisible par n'importe quel observateur. L'implémentation d'une **clé secrète partagée** (chiffrement symétrique) a permis de garantir :

- **La confidentialité :** Les données sont rendues illisibles (bruit numérique).
- **L'Intégrité :** Les paquets ne peuvent pas être modifiés durant le trajet.
- **L'Authenticité :** Seuls les serveurs possédant la clé peuvent communiquer.

En résumé, ce TP nous a appris à transformer une infrastructure réseau brute en un canal de communication sécurisé et optimisé, pilier fondamental de la cybersécurité moderne et du télétravail.